

ทำความเข้าใจเกณฑ์ประเมินโครงการ Data Engineering (6 ข้อ • 55 คะแนน)

เอกสารนี้ไว้ อ่านเพื่อทำความเข้าใจ เท่านั้น ไม่ได้วางแผนระบบ ไม่ได้ออกแบบ Evidence/Test Case และไม่ได้ประเมินว่าระบบได้กี่คะแนน

ข้อควรระวัง: คำอธิบายอิงจาก "ชื่อเกณฑ์" + หลักการ Data Engineering ทัวไป ไม่ได้อ่านบริกฉบับทางการ ถ้ากรรมการนิยามคำใดต่างไป ให้ยึดนิยามของกรรมการเป็นหลัก

สารบัญ

- แหล่งข้อมูลและปริมาณข้อมูล (10)
- จำนวนกระบวนการในการจัดการข้อมูล (15)
- ความครบถ้วนของ Data Extraction (10)
- ความครบถ้วนสมบูรณ์ของ Data Transformation (10)
- ความครบถ้วนและถูกต้องของ Data Loading (5)
- ประสิทธิภาพของการใช้ประโยชน์จากข้อมูล (5)
- ภาพรวมความสัมพันธ์ของทั้ง 6 ข้อ
- ตารางสรุป
- สิ่งที่ต้องเข้าใจให้ได้ก่อนทำระบบ
- ง่าย
- ภาคผนวก: คำศัพท์ที่มักสับสน

ภาพใหญ่ก่อนเริ่ม: Data Engineering คืออะไร

Data Engineering (วิศวกรรมข้อมูล) คืองานสร้าง "ท่อส่งข้อมูล" ที่พาข้อมูลดิบจากหลายที่ ไปสู่ที่ที่คนหรือระบบอื่นใช้งานได้ อย่างน่าเชื่อถือ

เปรียบเทียบกับ ระบบประปา

ระบบประปา	Data Engineering
แหล่งน้ำ (แม่น้ำ บ่อ)	Data Source แหล่งข้อมูล
สูบน้ำเข้าโรงงาน	Extraction / Ingestion ดึง/นำเข้าข้อมูล
กรองตะกอน ปรับค่า pH ใส่คลอรีน	Cleaning / Transformation ทำความสะอาด/แปลงข้อมูล
ตรวจสอบคุณภาพน้ำ	Data Quality ตรวจสอบคุณภาพข้อมูล
ส่งเข้าถึงเก็บ/ท่อประปา	Loading นำเข้าปลายทาง
คนเปิดก๊อกใช้น้ำ	Utilization นำข้อมูลไปใช้ประโยชน์

เกณฑ์ 6 ข้อที่ศึกษา คือการถามว่า "ตอนนี้ ต่อมาจากแหล่ง มีน้ำมากแค่ไหน (ข้อ 1) มีกี่ขั้นตอนระหว่างทาง (ข้อ 2) และ จุด (ข้อ 3) แปลง (ข้อ 4) ส่ง (ข้อ 5) ครบถ้วนถูกต้องไหม และสุดท้ายมีคนใช้น้ำจริงไหม (ข้อ 6)"

ข้อ 1 · จำนวนแหล่งข้อมูลและปริมาณข้อมูล (10 คะแนน)

1.1 Data Source (แหล่งข้อมูล) คืออะไร

Data Source = จุดที่ข้อมูลถูกสร้างขึ้นหรือถูกเก็บอยู่ ก่อนที่จะเข้ามาในท่อของเรา

คือคำตอบของคำถามว่า "ข้อมูลนี้มาจากไหน?"

ตัวอย่างในชีวิตจริง - ร้านค้าออนไลน์: ออเดอร์อยู่ในฐานข้อมูล, รายการสินค้าอยู่ในไฟล์ Excel/CSV จากซัพพลายเออร์, สถานะ การชำระเงินอยู่ที่ API ของธนาคาร, พฤติกรรมคลิกของลูกค้ามาเป็นสตรีมจากเว็บไซต์ - โรงพยาบาล: ระบบทะเบียนผู้ป่วย (DB), ผลแล็บจากเครื่องตรวจ (ไฟล์), นัดหมายจากแอปมือถือ (API)

แหล่งข้อมูล 1 แหล่ง $\neq$ 1 ไฟล์ เสมอไป ฐานข้อมูลหนึ่งตัวที่มี 20 ตาราง โดยทั่วไปนับเป็น 1 แหล่ง (แต่ 20 ตาราง) ส่วน CSV 3 ไฟล์ที่มาจากต่างระบบ อาจนับเป็น 3 แหล่งได้ วิธีนับขึ้นกับว่า "ต้นทางคือใคร/ระบบไหน/วิธีเข้าถึงอะไร"

1.2 Data Volume (ปริมาณข้อมูล) คืออะไร

Data Volume = ข้อมูลมีมากแค่ไหน คำตอบของคำถาม "ข้อมูลมีขนาดเท่าไร?"

มักตอบเป็น "จำนวนแถว/ระเบียน" และ/หรือ "ขนาดไฟล์/ขนาดที่เก็บ" (เช่น 2.4 ล้านแถว, 850 MB) อาจบอกอัตราเพิ่มด้วย (เช่น วันละ 50,000 แถว)

1.3 ประเภทของ Data Source

แบ่งได้หลายมุม

มุมที่ 1: ตามโครงสร้างของข้อมูล

ประเภท	ความหมาย	ตัวอย่าง
Structured (มีโครงสร้าง)	เป็นตารางแถว-คอลัมน์ ชัดเจน	ตารางใน MySQL, CSV
Semi-structured (กึ่งโครงสร้าง)	มีโครงสร้างแต่ยืดหยุ่น ซ้อนกันได้	JSON, XML
Unstructured (ไม่มีโครงสร้าง)	ไม่มีรูปแบบตายตัว	ข้อความอิสระ รูปภาพ PDF เสียง

มุมที่ 2: ตามวิธีเข้าถึง (มุมที่เกณฑ์ข้อนี้มักจะเน้น)

- ไฟล์ (File):** CSV, Excel, JSON, Parquet, XML
- ฐานข้อมูล (Database):** PostgreSQL, MySQL, SQL Server, MongoDB
- เว็บเซอร์วิส (REST API):** เรียก URL แล้วได้ JSON กลับมา
- สตรีม (Streaming):** ข้อมูลไหลมาต่อเนื่องแบบเรียลไทม์ เช่น Kafka, MQTT, WebSocket

- อื่น ๆ: Log file, Web scraping, Sensor/IoT, Message queue

1.4 เปรียบเทียบ CSV / Database / REST API / Streaming

	CSV	Database	REST API	Streaming
คืออะไร	ไฟล์ข้อความ คั่นค่าด้วยเครื่องหมาย (,)	ระบบเก็บข้อมูลที่ query ได้	บริการที่ตอบข้อมูลเมื่อถูกเรียก	ข้อมูลที่ไหลเข้ามาไม่หยุด
รูปแบบการได้ข้อมูล	อ่านไฟล์ทั้งก้อน	เขียน SQL ไปถาม	ส่ง HTTP request ได้ JSON	รอรับ event ตามที่เกิดขึ้น
ข้อมูลเปลี่ยนใหม่	นิ่ง (จนกว่าจะมีไฟล์ใหม่)	เปลี่ยนตลอด	เปลี่ยนตามเจ้าของ API	เปลี่ยนทุกวินาที
มีจุดสิ้นสุดไหม	มี (ไฟล์จบ)	มี (ตอน query)	มี (ที่ละหน้า)	ไม่มี (ไหลต่อเนื่อง)
ความเสี่ยงที่พบบ่อย	encoding ผิด, ตัวคั่นผิด	timeout, ข้อมูลเปลี่ยนระหว่างดึง	rate limit, pagination, schema เปลี่ยน	ช้า, ตกหล่น, มาซ้ำ/ผิดพลาด
ตัวอย่าง	ไฟล์ยอดขายรายเดือน	ตารางออเดอร์	API อัตราแลกเปลี่ยน	Sensor อุณหภูมิทุก 1 วิ

วิธีจำ: - **CSV** = กล่องจดหมายที่มีของอยู่แล้ว เปิดอ่านได้เลย - **Database** = ห้องสมุด ต้องไปถามบรรณารักษ์ (SQL) - **REST API** = โทรไปสั่งอาหาร เขาส่งมาให้ที่กล่อง (ที่ละหน้า) - **Streaming** = ยืนข้างสายพานลำเลียง ของมาเรื่อย ๆ ไม่รู้ว่าจะจบเมื่อไร

1.5 ทำไม "จำนวน Source" สำคัญใน Data Engineering

โลกจริงข้อมูลไม่ได้อยู่ที่เดียว (**data silo** = ข้อมูลแยกเป็นเกาะ) งานหลักของ DE คือ **รวมเกาะเหล่านั้นให้เป็นภาพเดียว** และแต่ละ Source เพิ่มความยาก:

- เข้าถึงคนละวิธี → ต้องเขียน **connector** (ตัวเชื่อม) คนละแบบ
- โครงสร้างต่างกัน → ชื่อคอลัมน์/ชนิดข้อมูลไม่ตรงกัน (cust_id vs customerID)
- รูปแบบต่างกัน → วันที่ 03/04/2025 กับ 2025-04-03, ปี พ.ศ. กับ ค.ศ.
- คุณภาพต่างกัน → บางแหล่งสะอาด บางแหล่งมีค่าว่างเยอะ
- จังหวะเวลาต่างกัน → ตัวหนึ่งอัปเดตรายวัน ตัวหนึ่งเรียลไทม์
- ต้องหา **key** ที่เชื่อมกันได้ (เช่น เลขออเดอร์ รหัสลูกค้า)

1.6 การมีหลาย Source สะท้อนความสามารถ DE อย่างไร

ระบบที่ดึงจาก Source เดียว (เช่น CSV 1 ไฟล์) แทบไม่ต้องแก้ปัญหา "บูรณาการ" ระบบที่รวมหลาย Source หลายประเภท สะท้อนว่า ผู้ทำ 1. เข้าใจ **วิธีเข้าถึงข้อมูลหลายแบบ** 2. จัดการ **ความต่างของโครงสร้าง/รูปแบบ** ได้ 3. ทำ **Data Integration** (การรวมข้อมูล) ได้จริง 4. สร้างท่อที่ **ใกล้เคียงงานจริง** ซึ่งแทบไม่เคยมีแหล่งเดียว

สังเกต: ไม่ได้แปลว่า "ยิ่งมากยิ่งดีแบบนับอย่างเดียว" แต่ **ความหลากหลายของประเภท** (ไฟล์ + DB + API + Stream) มักสะท้อนทักษะมากกว่ามี CSV 5 ไฟล์ที่หน้าตาเหมือนกัน

1.7 ปริมาณข้อมูลวัดอย่างไร

หน่วยวัด	หมายถึง	ตัวอย่าง
Record / Row (ระเบียน/แถว)	ข้อมูล 1 รายการ	ออเดอร์ 1 ใบ = 1 แถว
Column / Field (คอลัมน์/ฟิลด์)	คุณลักษณะของข้อมูล	ราคา, วันที่, ชื่อสินค้า
File Size / Storage Size (ขนาดไฟล์)	พื้นที่ที่ใช้เก็บ (KB, MB, GB, TB)	850 MB
Number of Files/Tables	จำนวนไฟล์/ตาราง	12 ตาราง
Growth Rate / Throughput	ความเร็วที่ข้อมูลเพิ่ม/ไหลผ่าน	+50,000 แถว/วัน, 2,000 events/วินาที

Records vs Rows vs Columns vs File Size ต่างกันอย่างไร

- **Row กับ Record** แทบเป็นสิ่งเดียวกันในตาราง คือ 1 รายการข้อมูล (ใช้ "record" กว้างกว่า เช่น JSON 1 object ก็เป็น 1 record แม้ไม่มีแถว)
- **Column** บอก "ความกว้าง" ของข้อมูล, **Row** บอก "ความยาว" ของข้อมูล
- **File size** บอกว่า "กินพื้นที่เท่าไร" และ **ไม่ได้เป็นสัดส่วนตรง ๆ กับจำนวนแถว**

ตัวอย่าง: ตาราง A มี 1,000,000 แถว × 3 คอลัมน์ตัวเลข อาจแค่ 30 MB ตาราง B มี 50,000 แถว × 40 คอลัมน์ที่มีข้อความยาว อาจ 300 MB → B แถวน้อยกว่ามาก แต่ **ใหญ่กว่า** ดังนั้นจึงควรรายงาน **หลายมุมประกอบกัน**

1.8 Small Data / Large Data คืออะไร

ไม่มีเส้นแบ่งตายตัว ขึ้นกับ "เครื่องมือที่ใช้จัดการ" แนวคิดคร่าว ๆ

ระดับ	ลักษณะ	เครื่องมือที่เพียงพอ
Small Data	เปิดทำงานในเครื่องเดียว/หน่วยความจำเดียวได้สบาย (Excel เปิดได้ ~ ไม่เกิน ~1 ล้านแถว)	Excel, pandas, SQLite
Medium	ระดับ GB ถึงหลายสิบล้าน GB ต้องมี DB จริงจัง	PostgreSQL, Spark local
Large / Big Data	เกินกว่าเครื่องเดียวรับไหว ต้องกระจายเครื่อง	Hadoop/HDFS, Spark cluster, Data warehouse

Big Data มักอธิบายด้วย "V" - **Volume** (ปริมาณ) · **Velocity** (ความเร็วที่เข้ามา) · **Variety** (ความหลากหลาย)

ข้อควรเข้าใจ: "Large" วัดที่ **เกินกว่าวิธีธรรมดาจะรับไหว** ไม่ได้วัดที่ตัวเลขเดียว ข้อมูล 5 ล้านแถวเป็น "เล็ก" ถ้าเครื่องรับได้สบาย แต่เป็น "ใหญ่" ถ้าต้องรันบนโน้ตบุ๊กเครื่องเล็ก

1.9 กรรมการักต้องการเห็นอะไรจากหัวข้อนี้ (เชิงแนวคิด)

- ระบุได้ชัดเจนว่าแต่ละ **Source** คืออะไร ชนิดอะไร มาจากไหน รูปแบบอะไร
- ตัวเลขปริมาณที่จริงและตรวจสอบได้ ไม่ใช่ประมาณลอย ๆ
- ความหลากหลายของชนิด **Source** ไม่ใช่แค่นับจำนวน
- ข้อมูล มีความหมายต่อโจทย์จริง (ไม่ใช่ข้อมูลของเล่น)

- (บ่อยครั้ง) ปริมาณมากพอที่ท่อข้อมูลมี "ความหมาย" ให้ต้องออกแบบจริงจัง

1.10 ตัวอย่างสถานการณ์จริง: ระบบวิเคราะห์ยอดขายร้านค้าออนไลน์

Source	ชนิด	ปริมาณ	ข้อสังเกต
ออเดอร์	PostgreSQL	1.2 ล้านแถว (วันละ +5,000)	ข้อมูลเปลี่ยนแปลงตลอด
แคตตาล็อกสินค้า	CSV จากซัพพลายเออร์	40,000 แถว, 12 MB	encoding ไทย อาจเพี้ยน
สถานะชำระเงิน	REST API ธนาคาร	~5,000 รายการ/วัน	ดึงทีละหน้า มี rate limit
คลิกบนเว็บ	Kafka stream	~300 events/วินาที	ไหลไม่หยุด อาจช้า

→ 4 Source, 3 วิธีเข้าถึง, ขนาดตั้งแต่หลัก MB จนถึงต่อเนื่อง: นี่คือน้ำตาของโจทย์ DE จริง

ข้อ 2 · จำนวนกระบวนการในการจัดการข้อมูล (15 คะแนน)

2.1 "กระบวนการจัดการข้อมูล" หมายถึงอะไร

คือ ขั้นตอนที่ข้อมูลต้องผ่านตั้งแต่ออกจากต้นทางจนถึงถูกใช้งาน ในวงการเรียกว่า **Data Pipeline (ท่อข้อมูล)** หรือ **Data Lifecycle (วงจรชีวิตข้อมูล)** แต่ละ "ขั้นตอน" คือ 1 กระบวนการ (process) ที่ทำอะไรบางอย่างกับข้อมูล

Source → Ingestion → Extraction → Validation → Cleaning → Transformation
→ Quality Check → Storage → Loading → Monitoring → Visualization/Analytics

หมายเหตุ: ชื่อ/การแบ่งขั้นตอนแต่ละตำราต่างกัน บางที่รวม Ingestion+Extraction เป็นอันเดียว บางที่แยก Storage กับ Loading ไม่ชัด สิ่งสำคัญคือ **เข้าใจหน้าที่** ไม่ใช่ท่องชื่อ

กรรมการที่ให้คะแนนข้อนี้ **น่าจะ** กำลังดู "ความครอบคลุม" ว่า ท่อข้อมูลมีขั้นตอนจัดการข้อมูลที่ครบ วงจรแค่ไหน เหมือนดูว่า "โรงงานน้ำประปามีกี่ขั้นตอน ครบไหม หรือแค่สูบแล้วส่งเลย"

2.2 อธิบายทีละกระบวนการ

① Source (แหล่งข้อมูล)

- **คืออะไร:** ต้นทางของข้อมูล (ดูข้อ 1)
- **ทำไมต้องมี:** ไม่มีต้นทางก็ไม่มีข้อมูล และต้นทางกำหนดวิธีดึง/ความเสี่ยงทั้งหมด
- **Input:** – • **Output:** ข้อมูลดิบที่รออยู่
- **ตัวอย่างง่าย:** ไฟล์ `sales.csv` บนเครื่อง
- **ตัวอย่างจริง:** ตารางออเดอร์ใน PostgreSQL ขององค์กร

② Ingestion (การนำข้อมูลเข้าระบบ)

- **คืออะไร:** การพาข้อมูลจากภายนอก **เข้ามาอยู่ในระบบของเรา** (รับ ขนส่ง วางลงที่פקซ์ชั่วคราว)
- **หน้าที่/ทำไมต้องมี:** เป็นประตูหน้าด่าน แยกข้อมูลต้นทางออกจากข้อมูลที่เราจัดการ ไม่ให้งานของเรารบกวนระบบต้นทาง
- **Input:** ข้อมูลที่ต้นทาง · **Output:** ข้อมูลดิบ (raw) ใน landing/staging area
- **ตัวอย่างง่าย:** ผู้ใช้อัปโหลดไฟล์ CSV เข้าระบบ
- **ตัวอย่างจริง:** Kafka consumer รับ event เก็บลง landing zone

③ Extraction (การดึง/สกัดข้อมูล)

- **คืออะไร:** การ **อ่าน/ดึงข้อมูลออกจากต้นทาง** ในรูปที่ใช้ต่อได้ (อธิบายละเอียดในข้อ 3)
- **ทำไมต้องมี:** ข้อมูลต้องออกจากต้นทางก่อนถึงจะแปลงได้
- **Input:** Source · **Output:** ชุดข้อมูลดิบที่ถูกอ่านแล้ว
- **ตัวอย่างง่าย:** `SELECT * FROM orders WHERE order_date = '2025-04-03'`
- **ตัวอย่างจริง:** Job ดึงออเดอร์ใหม่ทุกคืน ตั้งแต่ id ล่าสุดที่เคยดึง

④ Validation (การตรวจสอบความถูกต้องตามกฎหมาย)

- **คืออะไร:** การ **ตรวจสอบว่าข้อมูลตรงตามกฎหมายที่กำหนดหรือไม่** แบบตอบ ผ่าน/ไม่ผ่าน (ไม่แก้ข้อมูล แค่ตัดสินใจ)
- **ทำไมต้องมี:** กันข้อมูลเสียเข้าไปปนกับข้อมูลดี
- **Input:** ข้อมูลดิบ + กฎ (rule) · **Output:** ข้อมูลที่ผ่าน + รายการที่ไม่ผ่านพร้อมเหตุผล
- **ตัวอย่างง่าย:** อายุต้องอยู่ 0-120, อีเมลต้องมี @
- **ตัวอย่างจริง:** ตรวจสอบว่าทุกออเดอร์มี `customer_id` และ `total_price > 0`

⑤ Cleaning (การทำความสะอาดข้อมูล)

- **คืออะไร:** **แก้ไขหรือกำจัดข้อมูลที่ผิดปกติ** เช่น ค่าว่าง ขำ ตัวสะกดผิด ค่าผิดปกติ
- **ทำไมต้องมี:** "Garbage in, garbage out" ข้อมูลสกปรกให้ผลลัพธ์ห่วย
- **Input:** ข้อมูลดิบ · **Output:** ข้อมูลที่สะอาดขึ้น
- **ตัวอย่างง่าย:** ตัดช่องว่างหน้า-หลังชื่อ, ลบแถวซ้ำ
- **ตัวอย่างจริง:** รวมชื่อจังหวัด "กทม." "กรุงเทพ" "Bangkok" ให้เป็น "กรุงเทพมหานคร"

⑥ Transformation (การแปลงข้อมูล)

- **คืออะไร:** **เปลี่ยนรูปแบบ/โครงสร้าง/ความหมายของข้อมูล** ให้ตรงกับที่ต้องใช้ (ละเอียดในข้อ 4)
- **ทำไมต้องมี:** ต่างแหล่งต่างรูปแบบ ต้องทำให้ "พูดภาษาเดียวกัน" และสร้างค่าใหม่ที่มีประโยชน์
- **Input:** ข้อมูลที่ผ่านการตรวจ/ทำความสะอาด · **Output:** ข้อมูลในรูปแบบเป้าหมาย
- **ตัวอย่างง่าย:** แปลง `"03/04/2568"` เป็น `2025-04-03`
- **ตัวอย่างจริง:** Join ออเดอร์กับสินค้า แล้วคำนวณ `revenue = qty × price`

⑦ Quality Check (การตรวจคุณภาพข้อมูล)

- **คืออะไร:** การ **วัดคุณภาพของข้อมูลโดยรวม** หลังประมวลผล ดูหลายมิติ (Completeness ครบ, Accuracy ถูก, Consistency สอดคล้อง, Uniqueness ไม่ซ้ำ, Timeliness ทันเวลา, Validity ตรงรูปแบบ)

- ต่างจาก **Validation**: Validation = ตรวจ "ทีละค่า/ทีละแถว" ตามกฎ Quality Check = ดู "ภาพรวมของชุดข้อมูล" เป็นคะแนน/เปอร์เซ็นต์
- **ทำไมต้องมี**: ให้รู้ว่าข้อมูล ไร้ใจได้แค่ไหน
- **Input**: ข้อมูลหลังประมวลผล · **Output**: ตัวชี้วัด/รายงานคุณภาพ
- ตัวอย่างง่าย: "คอลัมน์ email มีค่าว่าง 2%"
- ตัวอย่างจริง: รายงานประจำวัน: แถวซ้ำ 0.1%, ผิด format 0.3%

⑧ Storage (การจัดเก็บข้อมูล)

- คืออะไร: การ เลือกและจัดวางที่เก็บข้อมูล ในแต่ละช่วง (raw, staging, processed)
- **ทำไมต้องมี**: ข้อมูลต้องอยู่ที่ไหนสักแห่งอย่างเป็นระบบ ปลอดภัย ค้นหาได้
- **Input**: ข้อมูลในแต่ละชั้น · **Output**: ข้อมูลที่ถูกเก็บในรูปแบบ/ตำแหน่งที่กำหนด
- ตัวอย่างง่าย: เก็บไฟล์ดิบไว้โฟลเดอร์ `raw/` ไฟล์ที่ประมวลผลแล้วไว้ `processed/`
- ตัวอย่างจริง: Data lake แบ่งโซน Bronze (ดิบ) / Silver (สะอาด) / Gold (พร้อมใช้)

⑨ Loading (การนำข้อมูลเข้าปลายทาง)

- คืออะไร: ส่งข้อมูลที่เตรียมเสร็จแล้วเข้าที่ปลายทาง ที่ผู้ใช้จะมาใช้ (ละเอียดในข้อ 5)
- **ทำไมต้องมี**: ข้อมูลที่แปลงเสร็จแต่ไม่ถึงปลายทาง คนใช้ก็เอาไปใช้ไม่ได้
- **Input**: ข้อมูลที่ประมวลผลแล้ว · **Output**: ข้อมูลอยู่ใน Destination
- ตัวอย่างง่าย: `INSERT` ข้อมูลลงตาราง
- ตัวอย่างจริง: โหลดตาราง `fact_sales` เข้า Data Warehouse ทุกคืน

⑩ Monitoring (การเฝ้าระวัง/ติดตาม)

- คืออะไร: การ เฝ้าดูว่าท่ทำงานปกติไหม ทั้งสถานะงาน เวลา error ปริมาณ และคุณภาพ
- **ทำไมต้องมี**: ท่จริงพังได้ทุกเมื่อ ถ้าไม่เฝ้า จะไม่รู้จนกว่าลูกค้าบ่น
- **Input**: log, ตัวชี้วัดระหว่างรัน · **Output**: สถานะ/การแจ้งเตือน/ประวัติ
- ตัวอย่างง่าย: บ่นที่กว่ารอบนี้ตั้งได้ก็แถว ใช้เวลาเท่าไร
- ตัวอย่างจริง: แจ้ง Slack เมื่อ job ล้มเหลวหรือจำนวนแถวลดผิดปกติ

⑪ Visualization / Analytics (การแสดงผล/วิเคราะห์)

- คืออะไร: การ นำข้อมูลมาทำให้คนเห็นและเข้าใจ หรือวิเคราะห์เชิงลึก (ละเอียดในข้อ 6)
- **ทำไมต้องมี**: ปลายทางของท่คือ "คนใช้ตัดสินใจ" ไม่ใช่แค่เก็บ
- **Input**: ข้อมูลที่โหลดแล้ว · **Output**: กราฟ แดชบอร์ด รายงาน insight
- ตัวอย่างง่าย: กราฟแท่งยอดขายรายเดือน
- ตัวอย่างจริง: แดชบอร์ดผู้บริหารดูยอดขายตามภาค

2.3 ตารางรวมทุกกระบวนการ

#	กระบวนการ	คำถามที่มันตอบ	Input → Output
①	Source	ข้อมูลอยู่ที่ไหน	— → ข้อมูลต้นทาง
②	Ingestion	พาเข้ามาอย่างไร	ต้นทาง → raw ในระบบ

#	กระบวนการ	คำถามที่มันตอบ	Input → Output
③	Extraction	อ่านออกมาครบไหม	ต้นทาง → ข้อมูลที่อ่านแล้ว
④	Validation	ผ่านกฎไหม	raw → ผ่าน/ไม่ผ่าน
⑤	Cleaning	สะอาดหรือยัง	raw → สะอาดขึ้น
⑥	Transformation	อยู่ในรูปที่ต้องการไหม	สะอาด → รูปเป้าหมาย
⑦	Quality Check	น่าเชื่อถือแค่ไหน	ข้อมูล → คะแนนคุณภาพ
⑧	Storage	เก็บอย่างเป็นระบบไหม	ข้อมูล → ที่เก็บ
⑨	Loading	ถึงปลายทางไหม	ข้อมูลพร้อม → Destination
⑩	Monitoring	ท่อยังปกติไหม	log/metric → แจ้งเตือน
⑪	Visualization	คนใช้ได้ไหม	ข้อมูล → insight

2.4 "จำนวนกระบวนการ" vs "จำนวน Feature" ต่างกันอย่างไร

	Process (กระบวนการ)	Feature (ฟีเจอร์)
มองจากมุมใคร	มุม ข้อมูล ว่ามันถูกทำอะไรบ้าง	มุม ผู้ใช้ ว่าทำอะไรกับระบบได้บ้าง
ตอบคำถาม	"ข้อมูลผ่านการจัดการขั้นไหนบ้าง?"	"ระบบมีความสามารถอะไรให้ใช้บ้าง?"
ลักษณะ	เป็น การกระทำกับข้อมูล (ดึง ตรวจสอบ แปลง โหลด)	เป็น ความสามารถที่ผู้ใช้เห็น/กดใช้ (อัปโหลด, ล็อกอิน, ส่งออก PDF, ค้นหา)
ตัวอย่าง	Extraction, Cleaning, Transformation, Loading	หน้า Login, ปุ่ม Export, หน้าแดชบอร์ด, ระบบแจ้งเตือน

ความสัมพันธ์: ไม่ใช่ 1:1 - 1 Feature อาจใช้ **หลาย Process** — ฟีเจอร์ "อัปโหลดไฟล์ยอดขาย" ซ่อน Ingestion + Validation + Cleaning + Loading ไว้ข้างหลัง - 1 Process อาจรองรับ **หลาย Feature** — Cleaning เดียวกัน ใช้ทั้งกับการอัปโหลดและกับการดึงจาก API - Feature บางอย่าง **ไม่ใช่ Process ของข้อมูลเลย** เช่น Login, เปลี่ยนธีมสีหน้าจอ, จัดการผู้ใช้

ตัวอย่างเทียบ: ร้านอาหาร - **Feature** = เมนูที่ลูกค้าสั่งได้ (ผัดไทย, ต้มยำ) - **Process** = ขั้นตอนในครัว (รับวัตถุดิบ ล้าง หั่น ปรง จัดจาน ตรวจสอบเสิร์ฟ) ร้านที่เมนูเยอะแต่ครัวไม่มีขั้นตอนตรวจสอบเลย ≠ ร้านที่ครัวเป็นระบบครบ

ทำไมสำคัญต่อการอ่านเกณฑ์ข้อนี้: กรรมาการที่ให้ 15 คะแนน "จำนวนกระบวนการ" กำลังดู **ความลึกของงานจัดการข้อมูลเบื้องหลัง** ไม่ใช่ว่า UI มีกี่ปุ่ม/กี่หน้า ระบบที่มี Feature เยอะแต่ข้อมูลผ่านแค่ "อ่านแล้วโชว์" จึงมี Process น้อย ขณะที่ระบบหน้าตาเรียบ ๆ แต่ข้อมูลผ่าน validate → clean → transform → quality check → load → monitor ก็มี Process ครอบคลุม

ข้อสังเกตเชิงเข้าใจ: การ "นับ" มักไม่ใช่แค่ใส่ชื่อขั้นตอนให้ครบ แต่ **แต่ละขั้นต้องทำงานกับข้อมูลจริง ๆ** (มี Input/Output ที่ชัด ไม่ใช่ขั้นตอนลอย ๆ)

ข้อ 3 · ความครบถ้วนของข้อมูลในช่วง Data Extraction (10 คะแนน)

3.1 Data Extraction คืออะไร

Data Extraction (การสกัด/ดึงข้อมูล) = การอ่านข้อมูลออกจากแหล่งต้นทาง ในรูปแบบที่ระบบของเราประมวลผลต่อไป

คือ "E" ใน **ETL** (Extract-Transform-Load)

3.2 Extraction ต่างจาก Ingestion อย่างไร

	Extraction	Ingestion
ขอบเขต	แคบ: การอ่าน/ดึงข้อมูลออกมา	กว้าง: ทั้งกระบวนการนำข้อมูลเข้าระบบ (รับ ขนส่ง ลงพัก)
มองจากฝั่ง	ฝั่ง ต้นทาง (ไปหยิบ)	ฝั่ง ปลายทางที่รับ (รับเข้ามา)
เปรียบเทียบ	ไปหยิบของที่ชั้นของร้านต้นทาง	ขนของกลับมาเก็บเข้าโกดังเรา
ตัวอย่าง	รัน SELECT, อ่านไฟล์, เรียก API	Kafka consumer รับ event, อัปโหลดไฟล์เข้า landing zone

หมายเหตุ: หลายตำราใช้สองคำนี้ **ปนกันหรือแทนกัน** (โดยเฉพาะในงานเล็ก ๆ) ถ้ากรรมการแยกคำ ก็คือ Extraction เน้น "ดึงออก", Ingestion เน้น "นำเข้า" เกณฑ์ข้อ 3 ใช้คำว่า Extraction → ให้โฟกัสที่ **"ดึงออกจากต้นทางได้ครบไหม"**

3.3 Extraction ทำงานอย่างไร (ภาพรวม)

- เชื่อมต่อต้นทาง (connect + authenticate)
 - กำหนดว่าจะดึงอะไร (ทั้งหมด หรือเฉพาะที่ใหม่/เปลี่ยน)
 - อ่านข้อมูล (query / อ่านไฟล์ / เรียก API / รับ message)
 - ถ้าเยอะ แบ่งอ่านเป็นก้อน (batch / chunk / page)
 - แปลงเป็นโครงสร้างที่ใช้ต่อไปได้ (parse)
 - วางลงพื้นที่พัก (staging)
 - บันทึกผลว่าดึงได้กี่รายการ ผิดพลาดตรงไหน

Full vs Incremental Extraction - Full (ดึงทั้งหมด): ดึงทั้งก้อนทุกครั้ง ง่ายแต่หนักเมื่อข้อมูลเยอะ - **Incremental (ดึงเฉพาะส่วนใหม่/ที่เปลี่ยน):** จำจุดล่าสุด (watermark เช่น `updated_at` หรือ `id`) แล้วดึงต่อ เบากว่าแต่เสี่ยงตกหล่นถ้าจุดอ้างอิงผิด

3.4 ดึงจากแต่ละ Source อย่างไร

CSV

- **ตัวอย่าง:** อ่านไฟล์ → แยกบรรทัด → แยกคอลัมน์ตามตัวคั่น → ตีความหัวตาราง

- ปัญหาที่พบ:
- **Encoding** (รหัสอักขระ): ไฟล์ภาษาไทยเก่ามักเป็น TIS-620/Windows-874 แต่อ่านเป็น UTF-8 → ตัวหนังสือเพี้ยน (à, ã, |à, Ñ...)
- ตัวคั่นต่าง (, ; tab) · มีเครื่องหมาย , ในข้อมูล (ต้องมี quote)
- บรรทัดเสีย จำนวนคอลัมน์ไม่ตรง · ขึ้นบรรทัดใหม่ในช่องข้อความ
- ไฟล์ใหญ่จนหน่วยความจำไม่พอ ต้องอ่านเป็นก้อน (chunk)
- รหัสนำหน้าด้วย 0 หาย (01000 → 1000) เพราะถูกอ่านเป็นตัวเลข

Database

- **ท้าวอย่างไร:** เชื่อมต่อ (JDBC/ODBC/driver) → ส่ง SQL → รับผลที่สะท้อน
- **ปัญหาที่พบ:**
- Connection timeout / query ช้าจนตัด
- ข้อมูลเปลี่ยนระหว่างดึง (มีแถวถูกเพิ่ม/ลบกลางทาง)
- Incremental ใช้ `updated_at` แต่ต้นทางลืมอัปเดต → ตกหล่น
- สิทธิ์ (permission) ไม่พอ เห็นไม่ครบทุกตาราง/แถว
- ดึงจากตัวหลักจนทำให้ระบบต้นทางช้า

REST API

- **ตัวอย่าง:** ส่ง HTTP GET (พร้อม token) → รับ JSON → แคะ (parse) → วนไปหน้าถัดไป
- **ปัญหาที่พบ:**
- **Pagination (แบ่งหน้า):** ลืมวนหน้า/วนไม่ครบ/นับหน้าผิด → ได้แค่หน้าแรก
- **Rate limit:** ถูกจำกัดจำนวนครั้ง (HTTP 429) ถ้าไม่ retry จะข้ามข้อมูล
- Timeout, token หมดอายุกลางทาง
- JSON ซ้อนกัน (nested) แคะไม่ครบ · API เปลี่ยนรูปแบบ (schema เปลี่ยน)

Streaming

- **ตัวอย่าง:** "สมัครรับ" (subscribe) หัวข้อ/ช่องทาง แล้วอ่านข้อความต่อเนื่อง Kafka จะมี **offset** (ลำดับที่อ่านถึง) กำกับ
- **ความเกี่ยวข้องกับ Extraction:** สตรีม **ไม่มีจุดสิ้นสุด** จึงไม่มีคำว่า "ได้ครบทั้งไฟล์" ความครบถ้วนจึงวัดเป็น **ช่วงเวลา/ช่วง offset** เช่น "ระหว่างเวลา 10:00–10:05 ส่งมา 12,000 event รับได้ 12,000 ไหม"
- **ปัญหาที่พบ:**
- **ช้า** (at-least-once delivery: ส่งซ้ำได้เพื่อกันหาย)
- **ตกหล่น** (consumer ล่ม ไม่ได้บันทึกว่าอ่านถึงไหน)
- **มาช้า/ผิดลำดับ** (late / out-of-order events)
- **Consumer ตามไม่ทัน** (lag สะสม)

3.5 "Extraction ที่ครบถ้วน" หมายถึงอะไร

ข้อมูลที่ถูกต้อง ครบในทกมิติ เหล่านี้:

มิติ	ครบถ้วนคือ
จำนวน	จำนวนระเบียบที่ได้ตรงกับที่ต้นทางมี (หรืออธิบายส่วนต่างได้)
ขอบเขต	ครบทุกไฟล์ ทุกหน้า ทุกพาร์ทิชัน ทุกตาราง ที่ตั้งใจจะดึง
ฟิลด์	ได้ทุกคอลัมน์ที่ต้องการ ไม่ขาดไม่เกิน
เนื้อหา	ค่าตรงต้นฉบับ ไม่เพี้ยน ไม่ถูกตัด
ความรับผิดชอบ	รายการที่ดึงไม่ได้ ถูกบันทึกและอธิบายได้ ไม่ใช่หายเงียบ ๆ

แก่นของคำว่า "ครบถ้วน" คือ **"นับบัญชีได้ลงตัว"** — ของที่เข้าบัญชี + ของที่ถูกปฏิเสธ(พร้อมเหตุผล) = ของทั้งหมดที่ต้นทาง

3.6 "Extraction ที่ไม่ครบถ้วน" เกิดจากอะไร

- Pagination ผิด / ข้ามหน้า
- Timeout / การเชื่อมต่อหลุดกลางทาง
- Rate limit แล้วไม่ retry
- แกวเสียถูก **ข้ามเจียบ ๆ** (parser ทั้งโดยไม่แจ้ง)
- Encoding ผิด ทำให้บางแถวอ่านไม่ได้
- เงื่อนไขกรอง (WHERE) หรือ watermark ผิด (off-by-one, time zone เหลื่อม)
- สิทธิ์เข้าถึงไม่ครบ
- ข้อมูลต้นทางเปลี่ยนระหว่างดึง
- Schema ต้นทางเปลี่ยน แล้ว mapping เดิมรองรับไม่ได้
- ข้อจำกัดของเครื่องมือ (จำกัดจำนวนแถวต่อครั้ง หน่วยความจำเต็ม)

3.7 Records หายได้อย่างไร (สถานการณ์ย่อ)

1. API คืนหน้าละ 100 รายการ แต่โค้ดวนแค่ 995 หน้า ทั้งที่มี 1,000 หน้า → หาย 500 รายการ
2. CSV มีแถวที่มี " " ไม่ปิด → parser ทั้งแถวนั้นและแถวถัดไปบางส่วน
3. ดึงเฉพาะ `updated_at > 10:00:00` แต่มีแถวที่ถูกแก้ตอน `10:00:00` พอดี → ตกหล่น
4. Consumer Kafka ล่มก่อน commit offset → ข้อความบางส่วนข้ามไปหรืออ่านซ้ำ

3.8 Data Corruption (ข้อมูลเสียหาย/ถูกบิดเบือน) คืออะไร

ข้อมูลที่ "มาถึง" แต่ค่า "ไม่ตรงกับต้นฉบับ" (นับจำนวนอาจครบ แต่เนื้อหาผิด)

ตัวอย่าง - ภาษาไทยกลายเป็นอักขระเพี้ยนจาก encoding ผิด - ตัวเลขถูกปิดทศนิยม `0.1 + 0.2` กลายเป็น `0.30000000000000004` หรือตัวเลขใหญ่ถูกย่อ `1.23E+10` - เลขศูนย์นำหน้าหาย `0812345678` → `812345678` - ข้อความยาวถูกตัดกลางทาง (truncate) - ไฟล์ดาวนโหลดไม่จบ ได้มาครึ่งเดียว - วันที่ถูกตีความผิด

จุดสำคัญ: **นับแถวครบ ≠ ข้อมูลถูกต้อง** จึงต้องดู "เนื้อหา" ด้วย ไม่ใช่แค่ "จำนวน"

3.9 Extraction Error คืออะไร

เหตุการณ์ผิดพลาดตอนอ่านข้อมูลจากต้นทาง เช่น ต่อเชื่อมไม่ได้ ยืนยันตัวตนไม่ผ่าน หมดเวลา สิทธิ์ไม่พอ แปลง/parse ไม่ได้ รูปแบบไฟล์ผิด

ประเด็นแนวคิด: error เกิดได้เสมอ สิ่งที่สำคัญคือ ระบบรับมืออย่างไร - ❌ เสียบ ๆ ทิ้งไป → ข้อมูลหายโดยไม่มีใครรู้ (อันตรายนี่สุด) - ✅ บันทึก (log) + ลองใหม่ (retry) + แยกรายการที่ผิดไปเก็บ (quarantine) + แจ้งเตือน

3.10 Schema ที่ไม่ตรงกันส่งผลอย่างไร

Schema (สคีมา/โครงสร้างข้อมูล) = ชื่อคอลัมน์ ลำดับ และชนิดข้อมูล ที่ระบบคาดหวัง Schema drift = ต้นทางเปลี่ยนโครงสร้างโดยที่เราไม่รู้

สิ่งที่เปลี่ยน	ผลที่เกิดขึ้น
คอลัมน์หายไป	error หรือกลายเป็นค่าว่างทั้งคอลัมน์
เปลี่ยนชื่อคอลัมน์ (price → unit_price)	ระบบหาไม่เจอ → ว่าง หรือ error
เพิ่มคอลัมน์ใหม่	ถูกเมินเสียบ ๆ หรือทำให้ mapping เลื่อน
ลำดับคอลัมน์สลับ (ถ้าอ่านตามตำแหน่ง)	ค่าไปอยู่ผิดช่อง แต่ไม่ error (อันตราย)
ชนิดข้อมูลเปลี่ยน (ตัวเลข → ข้อความ)	แปลงไม่ได้ บางแถวถูกทิ้ง

3.11 Extraction Completeness / Success Rate

- Extraction Success Rate (อัตราความสำเร็จ) = (จำนวนระเบียบที่ดึงสำเร็จ ÷ จำนวนระเบียบที่ต้นทางมี) × 100
- Extraction Completeness (ความครบถ้วน) มองกว้างกว่า คือ ครบทั้ง จำนวน ไฟล์ และเนื้อหา และรวมถึง การอธิบายส่วนที่ขาดได้

3.12 ตัวอย่าง: ต้นทาง 100,000 → ดึงได้ 99,500

ตัวเลข: $99,500 \div 100,000 = 99.5\%$ ขาดไป 500 รายการ (0.5%)

ควรตีความอย่างไร ตัวเลข 99.5% ไม่ได้บอกว่า "ผ่าน" หรือ "ไม่ผ่าน" โดยลำพัง ต้องถามต่อว่า 500 รายการนั้น "หายไปอย่างไร"

กรณี	สถานะ
500 รายการ ถูกปฏิเสธโดยตั้งใจและมีบันทึกเหตุผล (เช่น ผิดรูปแบบ) → $99,500 + 500 = 100,000$	ลงตัว ครบถ้วนในเชิงบัญชี
500 รายการ หายเสียบ ไม่มีใครรู้เหตุผล	ไม่ครบถ้วน เป็นปัญหา
ต้นทางมีการเปลี่ยนแปลงระหว่างดึง (ลบ 500 แถวพอดี)	อาจเป็นความต่างของ "เวลาที่นับ" ไม่ใช่การหายจริง

ต้องตรวจสอบอะไร 1. นับต้นทาง ณ ช่วงเวลาเดียวกับที่ดึง ได้ 100,000 จริงไหม (ไม่ใช่ตัวเลขเก่า) 2. ในผลที่ดึง มีแถวซ้ำไหม ถ้าซ้ำ 300 → ข้อมูลไม่ซ้ำจริงอาจเหลือ 99,200 (ขาดจริง 800) 3. ดูบันทึก error/ข้อมูลที่ถูกละเลย ว่ามี 500

รายการพร้อมเหตุผลใหม่ (ตัวเลขลงตัวใหม่) 4. หาว่า **500 รายการคือแถวไหน** โดยเทียบ key (primary key) ระหว่างต้นทางกับที่ดึงมา 5. **ดูรูปแบบของส่วนที่ขาด** ซึ่งช่วยชี้สาเหตุ - หายเป็นก้อนติดกัน / เลขกลม ๆ (500 พอดี) → สงสัย pagination หรือ batch ตกหน้าสุดท้าย - หายกระจัดกระจาย → สงสัยแถวเสีย/encoding/parse - หายเฉพาะช่วงเวลาหนึ่ง → สงสัย watermark/timzone/การล่มของ job 6. **เนื้อหาของ 99,500 ที่ได้ ถูกต้องไหม** (ไม่เพี้ยน) เพราะจำนวนครบ ≠ เนื้อหาถูก

ข้อ 4 · ความครบถ้วนสมบูรณ์ของการเปลี่ยนรูปข้อมูล (Data Transformation) (10 คะแนน)

4.1 Data Transformation คืออะไร

Data Transformation (การแปลง/เปลี่ยนรูปข้อมูล) = การเปลี่ยนข้อมูลจากรูปแบบ/โครงสร้างที่ได้มา ให้เป็นรูปแบบ/โครงสร้างที่ต้องการใช้งาน

คือ "T" ใน ETL

4.2 ทำไมต้อง Transform

- หลายแหล่งต่างรูปแบบ ต้องทำให้ "พูดภาษาเดียวกัน" ก่อนรวม
- ปลายทางต้องการรูปแบบเฉพาะ (ชนิดข้อมูล ชื่อคอลัมน์ ตามที่ตารางปลายทางกำหนด)
- สร้างค่าใหม่ที่มีความหมายทางธุรกิจ (เช่น กำไร = รายได้ - ต้นทุน)
- ทำให้วิเคราะห์ได้ง่าย/เร็วขึ้น (สรุปรวมไว้ล่วงหน้า)
- ข้อมูลจากต้นทางมักไม่พร้อมใช้ เพราะต้นทางสร้างมาเพื่อทำงานของมัน ไม่ใช่เพื่อวิเคราะห์

4.3 Transformation ต่างจากกระบวนการข้างเคียงอย่างไร

เปรียบเทียบกับ **ครว**: Validation = ตรวจวัตถุดิบ · Cleaning = ล้าง/ตัดส่วนเสีย · **Transformation** = หั่น ปรง จัดเป็นเมนู · Loading = เสิร์ฟ

	Transformation	Cleaning	Validation	Loading
ทำอะไร	เปลี่ยนรูป/โครงสร้าง/สร้างค่าใหม่	แก้ข้อมูลที่ผิด/สกปรก	ตรวจว่าตรงกฎ (ผ่าน/ไม่ผ่าน)	ส่งเข้าปลายทาง
แก้ข้อมูลใหม่	ใช่ (เปลี่ยนรูป)	ใช่ (แก้ความผิดพลาด)	ไม่แก้ (แค่ตัดสิน)	ไม่เปลี่ยนเนื้อหา แค่ย้าย
คำถาม	"ต้องเปลี่ยนเป็นอะไร"	"อะไรผิด แก้ได้ไหม"	"ผ่านกฎหรือไม่"	"สิ่งที่หมายใหม่"
ตัวอย่าง	แปลงวันที่, join, aggregate	เติมค่าว่าง, ลบซ้ำ, แก้พิมพ์ผิด	อายุ > 0 ?, email มี @ ?	INSERT ลง warehouse

หมายเหตุ: หลายตำราถือว่า **Cleaning** เป็นส่วนหนึ่งของ **Transformation** (เพราะก็เปลี่ยนข้อมูล) เส้นแบ่งคือ Cleaning = "แก้ที่ผิด" · Transformation = "เปลี่ยนให้เหมาะกับการใช้" ในทางปฏิบัติจึงเหลื่อมกัน ที่แน่ ๆ **Validation**

4.4 ตัวอย่าง Transformation ที่พบบ่อย

#	ชนิด	ความหมาย	Before → After
1	เปลี่ยน Data Type (ชนิดข้อมูล)	แปลงข้อความเป็นตัวเลข/วันที่ ฯลฯ	"1,250.50" (text) → 1250.50 (decimal)
2	เปลี่ยน Format วันที่	ปรับรูปแบบ/ปฏิทินให้เป็นมาตรฐาน	03/04/2568 → 2025-04-03
3	Normalize (ปรับให้เป็นรูปแบบมาตรฐาน/สเกลเดียวกัน)	ทำให้คำมีรูปแบบหรือช่วงเดียวกัน	ตัวพิมพ์ "BANGKOK" → "bangkok", หรือสเกลตัวเลข 0-1
4	Standardize (กำหนดให้ตรงมาตรฐานเดียว)	คำที่หมายถึงสิ่งเดียวกัน ให้เขียนแบบเดียวกัน	กทม., กรุงเทพฯ, Bangkok → กรุงเทพมหานคร
5	แปลงหน่วย	เปลี่ยนหน่วยวัด	2500 g → 2.5 kg, ฿ → USD
6	รวมข้อมูล (Merge/Concatenate)	รวมหลายคอลัมน์/ชุด	first + last → full_name
7	Split Column	แยกค่าหนึ่งเป็นหลายคอลัมน์	"12 ถนนสุขุมวิท, กรุงเทพฯ" → เลขที่, ถนน, จังหวัด
8	Rename Column	เปลี่ยนชื่อคอลัมน์ให้ตรงมาตรฐาน	cust_id → customer_id
9	Create Derived Column (คอลัมน์ที่สร้างจากคอลัมน์อื่น)	คำนวณค่าใหม่	revenue = qty × unit_price
10	Aggregation (การสรุปรวม)	รวมหลายแถวเป็นค่าสรุป	ออเดอร์รายใบ → ยอดขายรายวัน (SUM, COUNT, AVG)
11	Join Data	เชื่อมตารางด้วย key	ออเดอร์ + สินค้า → ออเดอร์พร้อมชื่อสินค้า
12	Remove Duplicate	ตัดแถวซ้ำ	2 แถวออเดอร์ #1001 เหมือนกัน → เหลือ 1
13	Handle Missing Value	จัดการค่าว่าง	เติมค่า (เช่น ค่าเฉลี่ย/ค่าเริ่มต้น), ใส่ Unknown, หรือตัดแถวทิ้ง

ข้อสังเกต: Remove Duplicate และ Handle Missing Value อยู่ "ก้ำกึ่ง" ระหว่าง Cleaning กับ Transformation ขึ้นกับว่าคุณจัดโครงสร้างเอกสารอย่างไร สิ่งสำคัญคือต้องรู้ว่า มันเป็นการเปลี่ยนข้อมูลที่ต้องถูกต้อง

4.5 "ความครบถ้วนสมบูรณ์ของ Data Transformation" หมายถึงอะไร

หมายถึง การแปลงข้อมูล "ครบ" และ "ถูก" ตามที่ตั้งใจ ทั้งก่อน แยกได้ 4 ด้าน

ด้าน	คำถาม
ครบทุกระเบียน	ทุกแถวที่ควรถูกแปลง ถูกแปลงหมดไหม (ไม่มีหลุด ไม่หายไปโดยไม่รู้เหตุผล)
ครบทุกฟิลด์/ทุกกฎ	ทุกคอลัมน์ที่ต้องแปลง ถูกแปลงหมดไหม ทุก rule ถูกนำไปใช้ไหม

ด้าน	คำถาม
ถูกต้อง	ค่าผลลัพธ์ตรงกับที่ควรเป็นไหม (rule เขียนถูก ไม่ใช่แค่ "รันผ่าน")
ตรงเป้าหมาย	ชนิดข้อมูล/ชื่อ/โครงสร้างหลังแปลง ตรงที่ปลายทางต้องการไหม

เส้นแบ่งสำคัญ: "รันจบไม่ error" ≠ "แปลงถูกและครบ" โค้ดที่รันผ่านอาจแปลงผิดหรือข้ามบางแถวอย่างเจียบ ๆ ก็ได้

4.6 จะรู้ได้อย่างไรว่า Transform ถูกต้อง (หลักคิดเชิงเข้าใจ)

- **นับบัญชี (Reconciliation):** แยกก่อน = แยกหลัง + แยกที่ตัดทิ้งอย่างมีเหตุผล (ลงตัวใหม่)
- **เทียบยอดรวม:** ผลรวมก่อน/หลังแปลงหน่วยต้องสอดคล้อง (เช่น ผลรวมกรัมก่อน ÷ 1000 = ผลรวมกิโลกรัมหลัง)
- **ตรวจชนิดและรูปแบบ:** ทุกค่าในคอลัมน์เป็นชนิด/รูปแบบที่ควร (เช่น วันที่ทุกแถว valid)
- **ตรวจช่วงค่า:** ค่าหลังแปลงอยู่ในช่วงที่สมเหตุสมผล (ราคาไม่ติดลบ)
- **เทียบตัวอย่างกับค่าที่คำนวณมือ:** สุ่มแถวมาเทียบกับคำตอบที่คิดเอง
- **หาแถวที่ "ไม่ถูกแต่":** มีแถวไหนที่ยังเป็นรูปแบบเดิมอยู่ไหม (เช่น วันที่ยังเป็น พ.ศ.)
- **ดูกรณีขอบ (edge case):** ค่าว่าง ค่าแปลก รูปแบบที่ไม่คาดคิด

4.7 ตัวอย่างเต็ม: Before → Transformation → After

Before (ข้อมูลดิบจากหลายแหล่ง)

order_id	order_date	price	qty	weight	province
1001	03/04/2568	"1,250.50"	2	2500 g	กทม.
1002	2025-04-04	"980"	1	1.2 kg	Bangkok
1003	3 เม.ย. 68	"abc"	3	–	กรุงเทพ
1001	03/04/2568	"1,250.50"	2	2500 g	กทม.

Transformation ที่ตั้งใจให้ทำ 1. `order_date` → รูปแบบ YYYY-MM-DD (ค.ศ.) 2. `price` → ตัวเลขทศนิยม 3. `weight` → หน่วยกิโลกรัมทั้งหมด 4. `province` → ชื่อมาตรฐาน 5. สร้าง `revenue = qty × price` 6. ตัดแถวซ้ำ

After (ผลที่ถูกต้อง)

order_id	order_date	price	qty	weight_kg	province	revenue
1001	2025-04-03	1250.50	2	2.5	กรุงเทพมหานคร	2501.00
1002	2025-04-04	980.00	1	1.2	กรุงเทพมหานคร	980.00
1003	2025-04-03	(ไม่ถูกต้อง → ส่งไปตรวจ/ค่าว่าง)	3	(ว่าง)	กรุงเทพมหานคร	(คำนวณไม่ได้)

สังเกต: แถวที่ 4 (ซ้ำ 1001) หายไป อย่างมีเหตุผล (ลบซ้ำ) แถว 1003 `price="abc"` แปลงไม่ได้ ต้อง ถูกบันทึกว่า แปลงไม่ได้ ไม่ใช่เจียบหายหรือกลายเป็น 0

4.8 ปัญหาที่อาจเกิดขึ้น

ปัญหา	ตัวอย่างจากตารางด้านบน
ข้อมูลหาย	แถว 1003 ถูกทิ้งเพราะ price แปลงไม่ได้ โดยไม่มีบันทึก → นับแล้วขาด
ข้อมูลผิดประเภท	price ยังเป็นข้อความ "1,250.50" เพราะลูกน้ำทำให้แปลงเป็นตัวเลขไม่ผ่าน หรือกลายเป็น 1 (อ่านแค่ก่อนลูกน้ำ)
Transformation Rule ผิด	แปลงปี พ.ศ. → ค.ศ. ผิด (สลับ 543 → ได้ปี 2568 ยังเหมือนเดิม) · 03/04 ตีความเป็น 4 มี.ค. แทน 3 เม.ย. (dd/mm vs mm/dd) · แปลง g → kg โดยคูณ 1000 แทนหาร
ข้อมูลบางส่วนไม่ได้ถูก Transform	rule วันที่รองรับแค่ dd/mm/yyyy พอเจอ 3 เม.ย. 68 ในแถว 1003 ก็ข้าม ปล่อยเป็นรูปแบบเดิมปนกับแถวอื่นที่แปลงแล้ว
ค่าหลัง Transformation ไม่ถูกต้อง	revenue ไม่ตรง (ใช้ราคาที่ยังไม่แปลงชนิด) · Join ผิด key ทำให้ชื่อสินค้าไม่ตรง · Join แล้วแถวเพิ่มซ้ำเพราะ key ไม่ unique

ข้อสังเกตสำคัญ: ปัญหาประเภท rule ผิด/ค่าผิด **ไม่ทำให้ระบบ error** แต่ผลผิด จึงอันตราย เพราะผู้ใช้ไม่รู้ตัว (เรียกว่า silent failure)

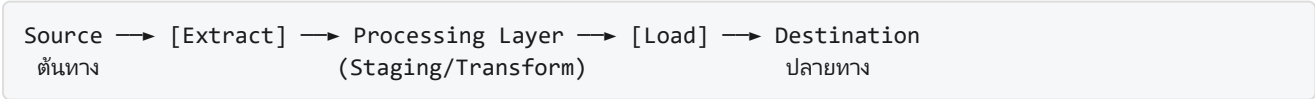
ข้อ 5 · ความครบถ้วนของความถูกต้องของการนำข้อมูลเข้าสู่ปลายทาง (Data Loading) (5 คะแนน)

5.1 Data Loading คืออะไร

Data Loading (การโหลด/นำข้อมูลเข้าปลายทาง) = การเขียนข้อมูลที่เตรียมเสร็จแล้ว ลงในที่เก็บปลายทางที่ผู้ใช้หรือระบบอื่นจะมาใช้งาน

คือ "L" ใน ETL

5.2 Loading อยู่ช่วงไหนของ Pipeline



คือ หลัง Transformation / Quality Check และ ก่อน ที่คนจะมาใช้ (Visualization/Analytics) (ถ้าเป็นรูปแบบ ELT จะโหลดข้อมูลดิบเข้าปลายทางก่อน แล้วค่อยแปลงในปลายทาง)

5.3 Source / Processing Layer / Destination ต่างกันอย่างไร

ชั้น	หน้าที่	ลักษณะ	เปรียบเทียบ (ครัว)
Source	ที่มาของข้อมูลดิบ	เราไม่ควรไปแก้ของเขา	ตลาด/ซัพพลายเออร์

ชั้น	หน้าที่	ลักษณะ	เปรียบเทียบ (ครัว)
Processing Layer	ที่ประมวลผล (extract/clean/transform)	ชั่วคราว เปลี่ยนแปลงได้	ครัว/โต๊ะเตรียมอาหาร
Destination	ที่เก็บผลสุดท้ายให้ใช้	ต้อง เสถียรและเชื่อถือได้	จานที่เสิร์ฟ/ตู้โชว์

5.4 Destination มีอะไรได้บ้าง

ปลายทาง	คืออะไร	เหมาะกับ	ตัวอย่าง
Database (ฐานข้อมูล)	เก็บแบบตารางเชื่อมกัน เน้นการทำธุรกรรมและค้นหา	ข้อมูลที่ระบบ/แอปใช้ตรง ๆ	PostgreSQL, MySQL
Data Warehouse (คลังข้อมูล)	ที่เก็บข้อมูล สะอาดและมีโครงสร้าง ออกแบบมาเพื่อ วิเคราะห์	รายงาน BI วิเคราะห์ข้ามหลายแหล่ง	BigQuery, Snowflake, Redshift
Data Lake (ทะเลสาบข้อมูล)	เก็บข้อมูล ดิบทุกรูปแบบ ราคาถูก ค่อยดีความภายหลัง	ข้อมูลปริมาณมาก/หลากหลายรูปแบบ, ML	S3/ADLS + Parquet
Elasticsearch	ระบบ ค้นหาและวิเคราะห์ข้อความ เก็บเป็น document (JSON)	ค้นหาเร็ว log analytics แดชบอร์ด (Kibana)	index ของ log, ค้นสินค้า
HDFS	ระบบไฟล์แบบกระจายของ Hadoop เก็บไฟล์ใหญ่บนหลายเครื่อง	Big data บน Hadoop/Spark ภายในองค์กร	คลัสเตอร์ Hadoop
Cloud Storage	ที่เก็บ ไฟล์/วัตถุ บนคลาวด์	เก็บไฟล์ สำรอง เป็นฐานของ data lake	Amazon S3, Google Cloud Storage, Azure Blob

จำง่าย: DB = ใช้งานธุรกรรม · Warehouse = วิเคราะห์ · Lake = เก็บทุกอย่างก่อน · Elasticsearch = ค้นหา · HDFS = ไฟล์ใหญ่กระจายเครื่อง · Cloud Storage = โกดังไฟล์บนคลาวด์

5.5 ประเภทของ Loading

แบ่งตาม "ปริมาณที่โหลด"

ประเภท	ความหมาย	ข้อดี / ข้อเสีย
Full Load (โหลดทั้งหมด)	ล้างปลายทาง/เขียนทับ แล้วโหลดทั้งหมดใหม่	ง่าย ตรงไปตรงมา · หนักเมื่อข้อมูลเยอะ
Incremental Load (โหลดเฉพาะส่วนใหม่/เปลี่ยน)	โหลดเฉพาะแถวที่เพิ่ม/แก้ไขตั้งแต่รอบก่อน (append หรือ upsert/merge)	เบาและเร็ว · ซับซ้อน เสี่ยงซ้ำ/ตกหล่น

แบ่งตาม "จังหวะเวลา"

ประเภท	ความหมาย	ตัวอย่าง
Batch Load (โหลดเป็นรอบ)	รวบรวมแล้วโหลดตามเวลา	ทุกคืนตอนตี 2
Streaming Load (โหลดต่อเนื่อง)	โหลดทีละ event/ไมโครแบตช์ เกือบเรียลไทม์	ธุรกรรมบัตรเครดิตเข้าระบบตรวจสอบจับทุจริต

ทั้งสองมุมมองผสมกันได้ เช่น "Incremental + Batch" (โหลดเฉพาะของใหม่ทุกคืน)

คำที่ควรรู้: **Idempotent** (รันซ้ำแล้วได้ผลเหมือนเดิม) — การโหลดที่ดี เมื่อรันซ้ำ (เช่น retry) ต้องไม่ทำให้ข้อมูลซ้ำ

5.6 "นำข้อมูลเข้าปลายทางที่ครบถ้วนและถูกต้อง" หมายถึงอะไร

หมายถึง สิ่งที่อยู่ในปลายทาง = สิ่งที่ตั้งใจส่งเข้าไป ทั้งจำนวนและเนื้อหา ไม่มีขาด ไม่มีเกิน ไม่เพี้ยน ไม่ผิดโครงสร้าง

5.7 ต้องตรวจสอบอะไรบ้าง

รายการ	ความหมาย	คำถาม
จำนวน Records	จำนวนแถวที่เข้าปลายทาง	ส่งเข้า 99,000 ปลายทางมี 99,000 ไหม
Schema	โครงสร้างปลายทาง	คอลัมน์/ลำดับ/ชื่อ ตรงกับที่กำหนดไหม
Data Type	ชนิดข้อมูลปลายทาง	วันที่ยังเป็นวันที่ ตัวเลขไม่กลายเป็นข้อความไหม
Data Integrity (ความถูกต้องของความสัมพันธ์/ข้อจำกัด)	ข้อมูลสอดคล้องและเชื่อมกันถูกต้อง	Primary key ไม่ซ้ำ, Foreign key ชี้ถูก, ไม่มี orphan record
Missing Records	แถวที่ควรอยู่แต่ไม่อยู่	ขาดหรือไม่ ขาดแถวไหน
Duplicate Records	แถวซ้ำที่ไม่ควรมี	โหลดซ้ำเพราะ retry ไหม
Failed Records	แถวที่โหลดไม่สำเร็จ	มีไหม ถูกเก็บไปไว้ไหน (dead-letter/reject table)
Loading Error	ข้อผิดพลาดตอนโหลด	เชื่อมต่อล้ม, constraint ไม่ผ่าน, ชนิดข้อมูลไม่ตรง, พื้นที่เต็ม — ถูกบันทึกไหม
เนื้อหาตรงกัน	ค่าไม่ถูกเปลี่ยน	ค่าในปลายทางเท่ากับค่าที่ส่งไหม

5.8 ตัวอย่าง: Processed Data → Loading → Destination

Processed Data (99,000 แถว, ยอดรวมขาย = 12,450,000 บาท)

| Loading

▼

Destination: ตาราง fact_sales ใน Data Warehouse

สถานการณ์ที่เจอได้

ผลที่ปลายทาง	แปลว่าอะไร
มี 99,000 แถว, ยอดรวม 12,450,000	✅ ตรงกัน
มี 98,700 แถว	ขาด 300 → ดู failed records: อาจ product_name ยาวเกินคอลัมน์ VARCHAR(50) จึง insert ไม่ผ่าน
มี 99,300 แถว	เกิน 300 → อาจโหลดซ้ำจาก retry หลังเครือข่ายหลุด (duplicate)

ผลที่ปลายทาง	แปลว่าอะไร
99,000 แถวตรง แต่ยอดรวม 12,300,000	จำนวนครบแต่ ค่าเพี้ยน (เช่น ทศนิยมถูกตัด, ชนิดข้อมูลปลายทางเป็นจำนวนเต็ม)

5.9 จะตรวจสอบอย่างไรว่าข้อมูลที่ปลายทางยังถูกต้อง

หลักคิด: เียบ "ก่อนโหลด" กับ "หลังโหลด" (Reconciliation / การกระหนยอด) และ ตรวจสอบเนื่อง

- เทียบจำนวนแถว ก่อน/หลัง
- เทียบค่ารวม (aggregate): SUM, MIN, MAX, COUNT DISTINCT ของคอลัมน์สำคัญ
- เทียบสั่มตัวอย่างแถว ทีละค่า
- ตรวจโครงสร้าง: schema / ชนิดข้อมูลของตารางปลายทางตรงกับที่ควร
- ตรวจความซ้ำ: นับ key ซ้ำ
- ตรวจ integrity: key เชื่อมกันครบ ไม่มี orphan
- ดูบันทึก failed/error: นับรวมแล้ว โหลดสำเร็จ + ล้มเหลว = ที่ส่งทั้งหมด ลงตัวไหม
- ตรวจ "ยังถูกต้องอยู่ไหม" ตามกาลเวลา: ข้อมูลปลายทางอาจถูกคนอื่นแก้/ลบ หรือ schema เปลี่ยน จึงต้องมีการเทียบซ้ำเป็นระยะ ไม่ใช่ตรวจแค่ครั้งเดียวตอนโหลด

ข้อ 6 · ประสิทธิภาพของการใช้ประโยชน์จากข้อมูล (5 คะแนน)

6.1 Data Utilization (การใช้ประโยชน์จากข้อมูล) คืออะไร

การนำข้อมูลที่ผ่านท่อมมาแล้ว ไปสร้างคุณค่าจริง ไม่ว่าจะเป็นช่วยให้ ตอบคำถาม ตัดสินใจ ติดตามผล หรือทำงานอัตโนมัติ

"ประสิทธิภาพ" (Effectiveness) = ทำแล้ว ได้ผลตามวัตถุประสงค์หรือไม่ (ต่างจาก "ประสิทธิภาพ" Efficiency ที่หมายถึง ทำเร็ว/ประหยัดแค่ไหน) ดังนั้นหัวข้อนี้ไม่ได้ถามแค่ว่า "ใช้ไหม" แต่ถามว่า "ใช้แล้วเกิดประโยชน์ตามเป้าไหม"

6.2 ข้อมูลที่ผ่านท่อมแล้ว ใช้ทำอะไรได้บ้าง

การใช้ประโยชน์	คืออะไร	ตัวอย่าง
Reporting (รายงาน)	สรุปข้อมูลเป็นรูปแบบรายงานตามรอบ	รายงานยอดขายประจำเดือน
Dashboard (แดชบอร์ด)	หน้าจอรวมตัวเลข/กราฟสำคัญ ดูได้ทันที	หน้าสรุปผู้บริหาร
Data Visualization (การแสดงผลข้อมูลเป็นภาพ)	กราฟ แผนที่ ตาราง ให้เห็นรูปแบบ	กราฟเส้นแนวโน้มยอดขาย
KPI (ตัวชี้วัดผลงานหลัก)	ตัวเลขชี้ความสำเร็จตามเป้า	ยอดขายเทียบเป้า 92%
Data Analysis (การวิเคราะห์)	ค้นหารูปแบบ/สาเหตุจากข้อมูล	ทำไมยอดขายภาคเหนือตก

การใช้ประโยชน์	คืออะไร	ตัวอย่าง
Business Intelligence (BI)	การใช้ข้อมูลช่วยธุรกิจตัดสินใจอย่างเป็นระบบ	Power BI, Tableau, Metabase
Data Quality Monitoring	ใช้ข้อมูล <i>เกี่ยวกับข้อมูล</i> ดูแลสุขภาพของท่อ	แดชบอร์ดอัตราข้อมูลครบ/ซ้ำ รายงาน
Machine Learning (การเรียนรู้ของเครื่อง)	ใช้ข้อมูลฝึกโมเดลทำนาย	ทำนายลูกค้าที่จะเลิกใช้บริการ
AI (ปัญญาประดิษฐ์)	ระบบฉลาดที่ใช้ข้อมูลทำงาน	แชทบอตตอบจากข้อมูลบริษัท, สรุปรายงานอัตโนมัติ
Decision Support (ระบบสนับสนุนการตัดสินใจ)	ให้ข้อมูล/ข้อเสนอแนะเพื่อช่วยคนตัดสินใจ	แนะนำจำนวนสต็อกที่ควรสั่ง

ข้อสังเกต: Data Quality Monitoring เป็นการใช้ประโยชน์ที่ **คนทำ DE มักมองข้าม** แต่เป็นการใช้งานข้อมูลโดยตรงกับงานของ DE เองที่สุด

6.3 "มีข้อมูล" กับ "สามารถใช้ประโยชน์จากข้อมูล" ต่างกันอย่างไร

	มีข้อมูล	ใช้ประโยชน์จากข้อมูลได้
สถานะ	ข้อมูลถูกเก็บอยู่	ข้อมูล ตอบคำถาม หรือ ช่วยตัดสินใจ ได้
คำถามที่ตอบ	"เรามีอะไรเก็บไว้"ไหม	"ข้อมูลนี้ช่วยให้เราทำอะไรได้"
คุณค่า	ยังเป็นต้นทุน (ค่าเก็บ ค่าดูแล)	เป็นมูลค่า
ตัวอย่าง	ตารางออเดอร์ 5 ล้านแถวในฐานข้อมูล ไม่มีใครเปิดดู	แดชบอร์ดที่ทำให้ผู้จัดการเห็นว่าสินค้า A ขาดสต็อกในภาคเหนือ แล้วสั่งเพิ่ม

เปรียบเทียบ: มีห้องสมุดเต็มไปด้วยหนังสือ แต่ไม่มีใครอ่าน/หาเล่มไม่เจอ → ห้องสมุดไม่ได้สร้างประโยชน์

ข้อมูลที่เก็บแต่ไม่มีใครใช้เรียกว่า **"Data Swamp / Data Graveyard"** (หนองข้อมูล/สุสานข้อมูล)

6.4 ตัวอย่างสถานการณ์จริง

Raw Sales Data	→	ออเดอร์ดิบ วันที่คนละรูปแบบ มีซ้ำ ราคาเป็นข้อความ
↓ Data Processing	→	validate / clean / transform / load
Clean Data	→	ตาราง fact_sales ที่สะอาด ชนิดข้อมูลถูกต้อง
↓ Analysis	→	สรุปยอดตามภาค/สินค้า/เดือน
Dashboard	→	กราฟยอดขาย + ตัวเลข KPI เทียบเป้า
↓		
Business Insight	→	"ยอดภาคเหนือลดลง 18% ในไตรมาส 3 เพราะสินค้า A ขาดสต็อก"
↓		
Decision / Action	→	สั่งเพิ่มสินค้า A ให้คลังภาคเหนือ

ตัวเลข 18% ในตัวอย่างเป็นเพียงภาพประกอบ

สังเกตว่า ถ้าข้อมูลต้นทางไม่ผ่านการจัดการ (วันที่ผิด ซ้ำ ขาด) ผลบนแดชบอร์ดจะผิด และ insight จะ พาให้ตัดสินใจผิด
→ นี่คือเหตุผลที่ข้อ 6 ผูกกับข้อ 3-5

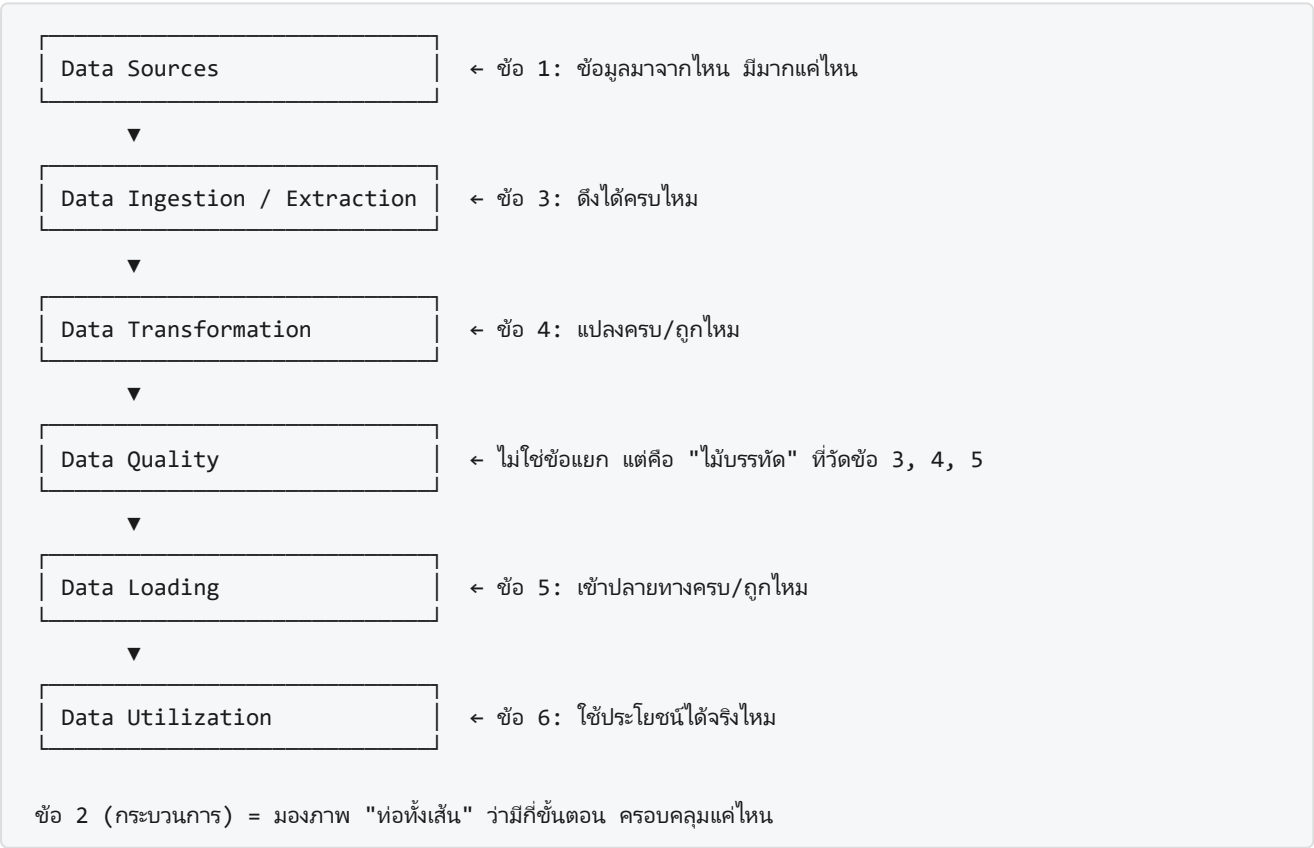
6.5 ทำไมขั้นสุดท้ายของ DE ไม่ควรจบแค่ "เก็บข้อมูล"

- ข้อมูลที่ไม่มีคนใช้ ไม่สร้างคุณค่า แต่ยังมีค่าใช้จ่ายในการเก็บและดูแล
- เป้าหมายของ DE คือสร้างข้อมูลที่ "พร้อมใช้" (analytics-ready) ไม่ใช่แค่ "ถูกเก็บ"
- การใช้งานจริงเป็น ตัวพิสูจน์ ว่าท่ทำงานถูก: ถ้าแดชบอร์ดผิด ย้อนไปหาสาเหตุในท่อได้
- ผู้ใช้ปลายทาง (นักวิเคราะห์ ผู้บริหาร โมเดล ML) ต้องการข้อมูลที่ เข้าถึงง่าย เข้าใจง่าย เชื่อถือได้ ทันเวลา
- ท่อที่เก็บอย่างเดียว เหมือนโรงงานผลิตแล้วเก็บของในโกดังโดยไม่เคยส่งขาย

"ประสิทธิภาพ" ในมุมมองข้อมูล มักพิจารณาจาก: ตอบคำถามที่ตั้งไว้ได้ไหม · ผู้ใช้เข้าใจและใช้งานได้ไหม · ข้อมูลทันเวลาพอไหม · ตัวเลขน่าเชื่อถือไหม · เกิดการตัดสินใจ/การกระทำจากข้อมูลจริงไหม

7 · ภาพรวมความสัมพันธ์ของทั้ง 6 ข้อ

7.1 Flow



7.2 แต่ละส่วนเชื่อมกับเกณฑ์ข้อใด

ส่วนของ Flow	เกี่ยวกับเกณฑ์ข้อ	อธิบาย
Data Sources	ข้อ 1	จุดเริ่มต้น: ก็แหล่ง ปริมาณเท่าไร
Ingestion / Extraction	ข้อ 3 (และนับเป็นกระบวนการในข้อ 2)	ดึงออกจากต้นทางครบไหม
Transformation	ข้อ 4 (และข้อ 2)	แปลงครบ ถูกไหม
Data Quality	ข้อ 3, 4, 5 (ตัวตัดสินความ "ครบถ้วน/ถูกต้อง") และเป็นหนึ่งในกระบวนการของข้อ 2	วัดว่าครบและถูกในแต่ละจุด
Loading	ข้อ 5 (และข้อ 2)	เข้าปลายทางครบถูกไหม
Utilization	ข้อ 6	ผลลัพธ์สุดท้าย: ใช้ประโยชน์จริงไหม
ทั้งหมด	ข้อ 2	ครอบคลุมก็กระบวนการ

เล่าเป็นเรื่อง: ข้อ 1 บอกว่า "วัตถุดิบมาจากไหนและเท่าไร" → ข้อ 3 ตรวจ "ซื้อมาครบไหม" → ข้อ 4 ตรวจ "ปรุงถูกสูตรและครบไหม" → ข้อ 5 ตรวจ "ส่งถึงโต๊ะครบและถูกไหม" → ข้อ 6 ถามว่า "ลูกค้ากินแล้วอึ้งและพอใจไหม" ส่วนข้อ 2 ถามว่า "ครัวนี้มีขั้นตอนก็อย่าง ครบวงจรแค่ไหน"

โครงสร้างของ 55 คะแนน - ข้อ 1 (10) = ความกว้าง/ขนาด ของงาน - ข้อ 2 (15) = ความครอบคลุมของกระบวนการ (ข้อที่หนักสุด) - ข้อ 3+4+5 (25) = ความครบถ้วนและถูกต้อง ของ E, T, L (แกนของ ETL) - ข้อ 6 (5) = คุณค่าปลายทาง

8 • ตารางสรุป

ข้อ	หัวข้อ	หมายถึงอะไร	กำลังวัดอะไร	ตัวอย่างใน Data Engineering
1 (10)	จำนวนแหล่งข้อมูลและปริมาณข้อมูล	ข้อมูลมาจากกี่แหล่ง ชนิดอะไร และมีมากเท่าไร	ความกว้างและขนาดของงาน บუნดาการหลายแหล่ง	DB ออเดอร์ 1.2 ล้านแถว + CSV สินค้า + REST API ชำระเงิน + Kafka คลิก
2 (15)	จำนวนกระบวนการในการจัดการข้อมูล	ขั้นตอนที่ข้อมูลผ่าน ตั้งแต่ต้นทางถึงใช้งาน	ความครอบคลุมของ Data Pipeline	Ingestion → Validation → Cleaning → Transformation → Quality Check → Storage → Loading → Monitoring
3 (10)	ความครบถ้วนของ Data Extraction	ดึงข้อมูลออกจากต้นทางได้ครบ ไม่เพี้ยน	อัตรา/ความครบถ้วนของการดึง และการอธิบายส่วนที่ขาด	ต้นทาง 100,000 ดึงได้ 99,500 → ต้องบอกได้ว่า 500 หายเพราะอะไร
4 (10)	ความครบถ้วนสมบูรณ์ของ Data Transformation	เปลี่ยนรูปข้อมูลครบทุกแถว/ทุกกฎ และถูกต้อง	ความถูกต้องและความครบของการแปลง	พ.ศ.→ค.ศ., "1,250.50"→1250.50, g→kg, สร้าง revenue

ข้อ	หัวข้อ	หมายถึงอะไร	กำลังวัดอะไร	ตัวอย่างใน Data Engineering
5	ความครบถ้วนและถูกต้องของ Data Loading	ข้อมูลเข้าปลายทางครบ ถูก ไม่ซ้ำ ไม่เพี้ยน	ผลที่ปลายทางตรงกับที่ส่ง	ส่ง 99,000 แถว ยอดรวม 12.45 ล้าน ปลายทางต้องได้เท่ากัน
6	ประสิทธิภาพของการใช้ประโยชน์จากข้อมูล	ข้อมูลถูกนำไปสร้างคุณค่าตามวัตถุประสงค์	ว่า "ใช้ประโยชน์ได้จริง" ไม่ใช่แค่เก็บ	ข้อมูลขายสะอาด → แดชบอร์ด KPI → ตัดสินใจเต็มสต็อก

9 · สิ่งที่ต้องเข้าใจให้ได้ก่อนทำระบบ

- ข้อ 1 · Source / Volume** - Source = **ต้นทางของข้อมูล** นับตามระบบ/วิธีเข้าถึง ไม่ใช่นับไฟล์ - Volume ต้องบอกได้หลายมุม (จำนวนแถว คอลัมน์ ขนาด อัตราเพิ่ม) - ความ "หลากหลาย" ของ Source สำคัญพอ ๆ กับจำนวน - "ใหญ่/เล็ก" ขึ้นกับว่าเครื่องมือธรรมดารับไหวไหม
- ข้อ 2 · Process** - Process = **สิ่งที่ทำกับข้อมูล** ไม่ใช่ปุ่มที่ผู้ใช้เห็น (นั่นคือ Feature) - รู้ว่าแต่ละขั้นมี **Input → Output** อะไร และ **ทำไมต้องมี** - Validation (ตรวจ ไม่แก้) ≠ Cleaning (แก้ผิด) ≠ Transformation (เปลี่ยนรูป) ≠ Quality Check (วัดภาพรวม) - ขั้นตอนต้องทำงานกับข้อมูลจริง ไม่ใช่ช้อลอย ๆ
- ข้อ 3 · Extraction** - "ครบถ้วน" = **นับบัญชีลงตัว** (ที่ได้ + ที่ถูกปฏิเสธพร้อมเหตุผล = ทั้งหมดที่ต้นทาง) - นับแถวครบ ≠ เนื้อหาถูก (Corruption) - แต่ละ Source พังคนละแบบ (CSV: encoding · DB: ข้อมูลเปลี่ยน · API: pagination/rate limit · Stream: ซ้ำ/ตกหล่น) - error ที่หายเงียบ ๆ อันตรายที่สุด
- ข้อ 4 · Transformation** - Transformation = **เปลี่ยนรูปให้พร้อมใช้** ไม่ใช่แค่ล้างข้อมูล - ต้อง "ครบ" (ทุกแถว ทุกกฎ) และ "ถูก" (ค่าตรงที่ควร) - รันผ่านไม่ error ≠ ถูกต้อง (silent failure) - ต้องรู้ว่าแถวไหนถูกตัด/แปลงไม่ได้ และเพราะอะไร
- ข้อ 5 · Loading** - ปลายทางต้อง **ตรงกับที่ส่ง** ทั้งจำนวน โครงสร้าง ชนิด และค่า - ต้องรู้ว่าแถวที่โหลดไม่สำเร็จไปอยู่ที่ไหน (Failed Records) - โหลดซ้ำต้องไม่ทำข้อมูลซ้ำ (Idempotent) - ตรวจแล้วต้องเทียบ "ก่อน vs หลัง" และตรวจซ้ำเป็นระยะ
- ข้อ 6 · Utilization** - "มีข้อมูล" ≠ "ใช้ประโยชน์ได้" - ปลายทางของ DE คือ **คนหรือระบบที่นำข้อมูลไปใช้** ไม่ใช่ที่เก็บ - ประสิทธิภาพ = ใช้แล้ว **ได้ผลตามเป้า** - ข้อ 6 พิสูจน์คุณภาพของข้อ 3-5 ไปด้วย (ท่อกับ → แดชบอร์ดผิด)

10 · ลำง่าย

1. **Source / Volume** = ข้อมูลมาจากไหน และมีมากแค่ไหน
2. **Process** = เราจัดการข้อมูลอย่างไร (กี่ขั้นตอน ครบวงจรไหม)
3. **Extraction** = ดึงข้อมูลมาได้ครบไหม
4. **Transformation** = แปลงข้อมูลได้ถูกและครบไหม
5. **Loading** = ส่งข้อมูลไปปลายทางได้ครบและถูกต้องไหม
6. **Utilization** = ข้อมูลที่ได้ เอาไปใช้ประโยชน์อะไรได้บ้าง

ประโยคเดียวรวม: "ข้อมูลมาจากไหน → ผ่านอะไรบ้าง → ดึงครบไหม → แปลงถูกไหม → ส่งถึงครบไหม → ใช้ประโยชน์ได้ไหม"

คำเตือนใจ 3 ข้อ - ครบ จำนวน ≠ ถูก เนื้อหา - รันไม่ error ≠ ได้ผลถูก - เก็บข้อมูลได้ ≠ ใช้ประโยชน์ได้

11 · ภาคผนวก: คู่คำที่มักสับสน

คู่คำ	ต่างกันอย่างไร (สั้น ๆ)
Extraction vs Ingestion	ดึง ออก จากต้นทาง vs นำ เข้า ระบบ (ตำราหลายเล่มใช้ปนกัน)
Validation vs Cleaning	ตรวจ ผ่าน/ไม่ผ่าน (ไม่แก้) vs แก้ ข้อมูลผิด
Cleaning vs Transformation	แก้ ความผิดพลาด vs เปลี่ยนรูปให้ เหมาะกับการใช้ (มักเหลื่อมกัน)
Validation vs Quality Check	ตรวจ ราคา/รายการ ตามกฎ vs วัด ภาพรวมชุดข้อมูล เป็นตัวชี้วัด
Transformation vs Loading	เปลี่ยน ข้อมูล vs ย้าย ข้อมูลเข้าปลายทาง
Normalize vs Standardize	ปรับให้อยู่ในรูป/สเกลเดียวกัน vs กำหนดให้ค่าที่หมายถึงสิ่งเดียวกันเขียนแบบเดียวกัน (สองคำนี้ใช้ปนกันบ่อย ดูนิยามในบริบท)
Process vs Feature	สิ่งที่ทำกับ ข้อมูล vs ความสามารถที่ ผู้ใช้ เห็น
Database vs Data Warehouse	เก็บเพื่อ ใช้งานระบบ/ธุรกรรม vs เก็บเพื่อ วิเคราะห์
Data Warehouse vs Data Lake	สะอาด มีโครงสร้าง vs ดิบ ทุกรูปแบบ
Full Load vs Incremental Load	โหลด ทั้งหมด ทุกรอบ vs โหลดเฉพาะ ส่วนใหม่/เปลี่ยน
Batch vs Streaming	ประมวลผลเป็น รอบ vs ต่อเนื่อง ทันที
ETL vs ELT	แปลง ก่อน โหลด vs โหลด ก่อน แล้วแปลงที่ปลายทาง
Record vs Row	แทบเหมือนกัน (record กว้างกว่า ใช้กับ JSON ได้)
Success Rate vs Completeness	สัดส่วน ที่ดึงสำเร็จ vs ความครบ ทั้งจำนวน ฟิลด์ เนื้อหา และบัญชีตัว
Data Corruption vs Missing Data	ข้อมูล มาแต่ผิด/เพี้ยน vs ข้อมูล ไม่มา
มีข้อมูล vs ใช้ประโยชน์จากข้อมูล	เก็บอยู่ vs สร้างคุณค่า/ช่วยตัดสินใจได้
Effectiveness vs Efficiency	ได้ผลตามเป้า (ประสิทธิผล) vs ทำเร็ว/ประหยัด (ประสิทธิภาพ)